

ICS2121, Métodos de Optimización

Una muy breve introducción a la Programación Dinámica

Lo que veremos

- Motivación con el “Problema de la Mochila”
- La lógica de la Programación Dinámica (PD)
- Algunos ejemplos
- Programación Dinámica en general y su poder para abordar problemas complejos.
- Habrá un video complementario y notas de clases con más ejemplos y temas adicionales.

Motivación: el Problema de la Mochila

- Lo recuerdan, ¿verdad?
- Recordemos el modelo básico:
- x_j es el número de unidades del item j a llevar en la mochila
- a_j es el volumen unitario del item j .
- e_j es el valor del item tipo j .
- K es el volumen disponible.

$$\begin{aligned} z^* = \text{máx} \quad & \sum_{j=1}^n e_j x_j \\ \text{s.a.} \quad & \sum_{j=1}^n a_j x_j \leq K \\ & x_j \geq 0 \text{ entero } j = 1, \dots, n \end{aligned}$$

- Es un problema de Programación Entera “como cualquier otro”, ¿o no?

Motivación: el Problema de la Mochila

- Vamos a ilustrar otra forma de abordarlo: descomponer el problema en “etapas” de decisión.
- Supongamos que vamos llenando la mochila “de a un tipo de ítem a la vez”.
- En la primera etapa de llenado decidimos cuántas unidades del ítem 1 llevaremos.
- Esta decisión impacta las alternativas disponibles para la segunda etapa: cuántas unidades llevar de 2 y así...
- Hay un “estado” del “sistema”: la capacidad disponible de la mochila en una etapa dada, la cual depende de las decisiones anteriores.

Motivación: el Problema de la Mochila

- Usamos estas “etapas” para descomponer el problema.
- Reescribamos el problema en forma extendida:

$$\begin{aligned} z^* = \max \quad & e_1x_1 + e_2x_2 + \cdots + e_nx_n \\ \text{s.a.} \quad & a_1x_1 + a_2x_2 + \cdots + a_nx_n \leq K \\ & x_j \geq 0 \text{ entero } j = 1, \dots, n \end{aligned}$$

Motivación: el Problema de la Mochila:

- Tenemos, entonces:

$$z^* = \max_{x_1 \geq 0, \text{ entero}} e_1 x_1 + \left\{ \begin{array}{l} \max \sum_{j=2}^n e_j x_j \\ \text{s.a.} \sum_{j=2}^n a_j x_j \leq K - a_1 x_1 \\ x_j \geq 0 \text{ entero } j = 2, \dots, n \end{array} \right\}$$

- Observamos que el problema “interior” está subordinado a la decisión en el índice 1, o, en la etapa 1.

Motivación: el Problema de la Mochila

- Definamos:

$$\begin{aligned} G_i(b) = \max \quad & \sum_{j=i}^n e_j x_j \\ \text{s.a.} \quad & \sum_{j=i}^n a_j x_j \leq b \\ & x_j \geq 0 \text{ entero } j = i, \dots, n \end{aligned}$$

- Teníamos:

$$z^* = \max_{x_1 \geq 0, \text{ entero}} e_1 x_1 + \left\{ \begin{array}{l} \max \quad \sum_{j=2}^n e_j x_j \\ \text{s.a.} \quad \sum_{j=2}^n a_j x_j \leq K - a_1 x_1 \\ x_j \geq 0 \text{ entero } j = 2, \dots, n \end{array} \right\}$$

- ¿Cómo escribimos z^* usando la función G_i ?

Motivación: el Problema de la Mochila

- Tenemos, entonces, la siguiente relación:

$$z^* = G_1(K) = \max_{x_1 \geq 0, \text{ entero}} \{e_1 x_1 + G_2(K - a_1 x_1)\}$$

- Más aún, podemos definir una restricción explícita para $x_1 \dots$
- Y en forma más general, para cada etapa $i = 1, \dots, n - 1$:

$$G_i(b) = \max_{0 \leq x_i \leq \left\lfloor \frac{b}{a_i} \right\rfloor, \text{ entero}} \{e_i x_i + G_{i+1}(b - a_i x_i)\}$$

- que se resuelve recursivamente...

Motivación: el Problema de la Mochila

- Cada etapa depende de la siguiente, hasta la etapa n :

$$G_n(b) = \max_{0 \leq x_n \leq \left\lfloor \frac{b}{a_n} \right\rfloor, \text{ entero}} e_n x_n$$

que es la que realmente se puede calcular...

- Y de ahí nos devolvemos por las recursiones...
- para finalmente calcular $G_1(K)$.
- Todo el cálculo se puede hacer, en el peor caso, en $O(nK)$ evaluaciones totales (iteraciones) y muchas combinaciones no es necesario evaluarlas.

Ejemplo:

- Consideremos el siguiente problema:

$$\begin{array}{ll}\text{máx} & 3x_1 + 6x_2 + x_3 \\ \text{s.t.} & 5x_1 + 3x_2 + 2x_3 \leq 10 \\ & x_i \geq 0 \text{ entero}, i = 1, 2, 3\end{array}$$

- En lo que sigue se muestra la secuencia de cálculos:

Ejemplo:

$$\begin{aligned} z^* = G_1(10) = \max \quad & 3x_1 + 6x_2 + x_3 \\ \text{s.a.} \quad & 5x_1 + 3x_2 + 2x_3 \leq 10 \\ & x_i \geq 0 \text{ entero}, i = 1, 2, 3 \end{aligned}$$

$$G_1(10) = \max_{0 \leq x_1 \leq 2, \text{ entero}} \{3x_1 + G_2(10 - 5x_1)\}$$

$$G_1(10) = \max\{0 + G_2(10), 3 + G_2(5), 6 + G_2(0)\}$$

$$\begin{aligned} G_2(10) = \max \quad & 6x_2 + x_3 \\ \text{s.a.} \quad & 3x_2 + 2x_3 \leq 10 \\ & x_i \geq 0 \text{ entero}, i = 2, 3 \end{aligned}$$

$$\begin{aligned} G_2(5) = \max \quad & 6x_2 + x_3 \\ \text{s.a.} \quad & 3x_2 + 2x_3 \leq 5 \\ & x_i \geq 0 \text{ entero}, i = 2, 3 \end{aligned}$$

Ejemplo:

$$\begin{aligned} G_2(10) = \max \quad & 6x_2 + x_3 \\ \text{s.a.} \quad & 3x_2 + 2x_3 \leq 10 \\ & x_i \geq 0 \text{ entero}, i = 2, 3 \end{aligned}$$

$$G_2(10) = \max_{0 \leq x_2 \leq 3, \text{ entero}} \{6x_2 + G_3(10 - 3x_2)\}$$

$$G_2(10) = \max\{0 + G_3(10), 6 + G_3(7), 12 + G_3(4), 18 + G_3(1)\}$$

$$\begin{aligned} G_2(5) = \max \quad & 6x_2 + x_3 \\ \text{s.a.} \quad & 3x_2 + 2x_3 \leq 5 \\ & x_i \geq 0 \text{ entero}, i = 2, 3 \end{aligned}$$

$$G_2(5) = \max_{0 \leq x_2 \leq 1, \text{ entero}} \{6x_2 + G_3(5 - 3x_2)\}$$

$$G_2(5) = \max\{0 + G_3(5), 6 + G_3(2)\}$$

Ejemplo:

- Finalmente hay que resolver el problema de la última etapa:

$$\begin{aligned} G_3(b) = \max \quad & x_3 \\ \text{s.a.} \quad & 2x_3 \leq b \\ & x_3 \geq 0 \text{ entero} \end{aligned}$$

- La siguiente tabla muestra los valores requeridos:

b	$G_3(b)$	x_3
1	0	0
2	1	1
4	2	2
5	2	2
7	3	3
10	5	5

Ejemplo:

- Y ahora nos “devolvemos”. Esto se llama “backtracking”:

$$G_2(5) = \max\{0 + G_3(5), 6 + G_3(2)\} = \max\{0 + 2, 6 + 1\} = 7$$

$$x_2 = 1, x_3 = 1$$

$$G_2(10) = \max\{0 + G_3(10), 6 + G_3(7), 12 + G_3(4), 18 + G_3(1)\}$$

$$G_2(10) = \max\{0 + 5, 6 + 3, 12 + 2, 18 + 0\} = 18$$

$$x_2 = 3, x_3 = 0$$

$$G_1(10) = \max\{0 + G_2(10), 3 + G_2(5), 6 + G_2(0)\}$$

$$G_1(10) = \max\{0 + 18, 3 + 7, 6 + 0\} = 18$$

$$x_1 = 0, x_2 = 3, x_3 = 0$$

La lógica de la Programación Dinámica

- La PD fue desarrollada por Richard Bellman alrededor de 1953. Estos son los principios:
- La búsqueda de soluciones factibles es un proceso que ocurre en **etapas** y el costo (o beneficio) total es la suma de los costos (o beneficios) de las **decisiones (o acciones)** individuales.
- El **estado del sistema** en una etapa depende de las decisiones en las etapas anteriores.
- La dinámica del sistema está descrita por una **función de transición** y el costo (o beneficio) de cada transición es el costo asociado a la decisión.
- Se debe escribir una recursión del costo (o beneficio) óptimo desde la etapa actual hasta un estado de destino.
- Un problema que se formule mediante Programación Dinámica cumple con:
- **El Principio de Optimalidad de Bellman:** Supongamos tenemos una solución óptima al problema, $x_1, x_2, \dots, x_N$. Sea $k < N$ una etapa intermedia del problema. Entonces, la solución parcial de la etapa 1 a la k , $x_1, x_2, \dots, x_k$ es óptima para el problema correspondientes a las etapas de la 1 a la k .

El esquema general de Programación Dinámica

- En términos más generales, se definen etapas, indexadas por k , el estado del sistema en la etapa k , s_k , y las “políticas”, “acciones”, o “controles” (la solución), x_k .
- Se define la función $J_k(s_k)$ que corresponde al costo (o beneficio) del sistema desde la etapa k hasta el final de horizonte, cuando el estado en la etapa k es s_k .
- g_k es la función de costos de la etapa k y f_k es la función de transición, indica cómo cambia el estado según las decisiones.

- Se cumple, entonces, la **Ecuación de Bellman** (la recursión):

$$J_k(s_k) = \min_{x_k \in U_k(s_k)} \{g_k(s_k, x_k) + J_{k+1}(f_k(s_k, x_k))\}$$

- Se define una condición de borde $J_N(s_N) = g_N(s_N)$.
- Objetivo: calcular $J_0(s_0)$.
- Es fácil ver que el Problema de la Mochila calza con este esquema y los ejemplos que veremos también.

Ejemplos: Reemplazo de Equipos

- En muchos ámbitos industriales hay equipos de alto costo...
- Durante su vida útil requieren inversión en mantenimiento.
- Eventualmente eso se hace demasiado caro.
- El equipo suele tener un valor residual.
- Tenemos que decidir cuándo reemplazar.



(imágenes de wikipedia.com)

Reemplazo de Equipos:

- Supongamos un horizonte de N años.
- Se reemplaza al comienzo del periodo y no hay reemplazo en el último año.
- El “estado” será la edad del equipo al comienzo de un período determinado. Denotemos por e_t la edad al comienzo del periodo t .
- Sea I la inversión requerida para un equipo nuevo.
- Sea $S(e)$ el valor residual de un equipo de edad e .
- Sea E_0 la edad inicial del equipo.
- Sea $c_t(e)$ el costo operacional de un equipo de edad e en el periodo t .
- En el último año, se incurre en el costo de operación y el equipo se “vende” por su valor residual.

Reemplazo de Equipos:

- Sea $f_t(e_t)$ el costo total desde el periodo t hasta el final del horizonte para un equipo de edad e_t . Tenemos que calcular $f_1(E_0)$
- Construyamos la recursión...
- Resumiendo, la recursión es:

$$\begin{aligned}f_t(e_t) &= \min\{I - S(e_t) + c_t(0) + f_{t+1}(1), c_t(e_t) + f_{t+1}(e_t + 1)\} \\f_N(e_N) &= c_N(e_N) - S(e_N)\end{aligned}$$